

Documentation de gestion de projet — Vite & Gourmand

TP Développeur Web et Web Mobile (Studi) · Antoine Banzet · 2026

1. Contexte et contraintes

Projet réalisé **en autonomie**, en parallèle d'un emploi et de la montée en compétences de la formation. Il s'agit de mon premier projet informatique d'envergure. Deux contraintes ont structuré toute l'organisation :

- **Le temps** — le développement devait commencer, et une bonne partie être livrée, avant même la fin du cursus d'apprentissage.
- **Le niveau** — chaque choix technique devait rester compréhensible, corrigéable et défendable à l'oral ; pas de sur-ingénierie.

2. Méthodologie

J'ai privilégié une démarche **itérative et incrémentale** (esprit Agile) plutôt qu'une gestion prédictive type cycle en V ou cascade. Pour un développeur seul qui découvre plusieurs technologies en même temps, un plan figé d'emblée aurait été irréaliste : les défis techniques imposent des ajustements permanents. L'application a donc été construite brique par brique, chaque partie étant testée au fur et à mesure de sa production.

3. Suivi et découpage en sprints

Le séquençement s'est appuyé sur la **progression pédagogique de la formation** : les briques techniques ont été abordées dans l'ordre du cursus (intégration HTML/CSS → JavaScript → PHP et API → base relationnelle → base NoSQL → sécurité → déploiement). Ce découpage a été formalisé sur **GitHub** : 7 jalons (*milestones*) correspondant aux sprints, chacun décliné en tâches (*issues*). L'historique Git (commits datés, branches de fonctionnalités) en constitue la trace détaillée.

Sprint	Objectif	Livrable
1	Analyse de l'énoncé, maquettage, modélisation des données	Wireframes, MCD/MLD, schéma SQL
2	Intégration front-end statique	Toutes les pages HTML/CSS, responsive
3	API : socle, authentification, catalogue	Connexion/inscription, endpoints menus & filtres
4	Commandes et espaces de gestion	Tunnel de commande, dashboards employé et admin
5	Base NoSQL et statistiques	Comptage MongoDB, graphique « commandes par menu »
6	Relecture et simplification	Retrait des fonctions trop complexes, nettoyage du code
7	Tests, sécurité, accessibilité, déploiement	Grille OWASP, conformité RGPD/RGAA, mise en ligne

4. Gestion des priorités et arbitrages

Plusieurs idées d'évolution ont été volontairement écartées pour tenir les délais et garantir un code stable :

- **Calendrier de réservation interactif** — prévu au départ pour le choix de la date de livraison. Le rendre réellement fiable (disponibilité réelle des créneaux, gestion des conflits, capacité par jour, jours fériés) dépassait ce que je pouvais garantir dans le temps restant. Remplacé par un simple champ `<input type="date">` validé côté serveur — entièrement fonctionnel et conforme au besoin de l'énoncé.

- **Planning avancé du personnel** et **page dédiée à la gestion du matériel** — hors périmètre du cahier des charges, abandonnés au profit d'un recentrage strict sur les fonctionnalités demandées.

Principe retenu : livrer une fonctionnalité plus simple mais maîtrisée et explicable, plutôt qu'un composant plus impressionnant mais dont la fiabilité resterait incertaine.

5. Workflow Git

Le dépôt suit une organisation à deux branches longues et des branches de fonctionnalités :

- **main** — code stable ; ne reçoit que des fusions de **develop** validées, marquées comme *releases*.
- **develop** — branche d'intégration.
- **feature/*** — une branche par fonctionnalité, issue de **develop**, fusionnée dans **develop** après test avec `--no-ff` (le commit de fusion conserve la trace de la branche).

Les messages de commit sont atomiques et descriptifs (préfixes **feat**, **fix**, **chore**, **docs**). Deux *releases* jalonnent le projet : la première à la fin de la construction initiale, la seconde après la phase de renforcement (refonte de la navbar, sécurité du cookie de session, conformité RGPD, finitions).

6. Outils

Besoin	Outil
Éditeur de code	Visual Studio Code
Environnement local	XAMPP (Apache, PHP, MySQL) + phpMyAdmin
Versionnage	Git / GitHub
Suivi de sprints	GitHub — jalons (milestones) et issues
Maquettage	Figma / Balsamiq (wireframes)
Base NoSQL	MongoDB Atlas
Hébergement	Render (application), Aiven (MySQL), Brevo (emails)

7. Difficultés rencontrées et enseignements

- **Gérer le temps en développant et en apprenant en même temps** — résolu par le découpage en sprints calé sur l'ordre des cours, qui a évité de coder « à l'aveugle ».
- **Le déploiement a révélé des bugs invisibles en local** — module Apache manquant, différences de mode SQL entre MySQL local et l'hébergeur, redirection cassée derrière le proxy, conflit de configuration CORS, et emails qui ne partaient plus une fois en production : Brevo rejette silencieusement tout envoi dont l'adresse d'expédition n'est pas vérifiée dans le compte, et le domaine fictif `viteetgourmand.fr` ne pouvait pas l'être. Diagnostiqué via le journal d'activité de Brevo, corrigé en utilisant mon adresse personnelle vérifiée (`antoineaban@gmail.com`) comme expéditeur technique. Chaque environnement a ses propres tolérances : un code qui marche en local n'est pas une garantie qu'il marchera en production.
- **Une API fonctionnelle ne suffit pas** — il a fallu vérifier que chaque route disposait bien d'un point d'entrée réellement utilisable dans l'interface.

8. Utilisation de l'intelligence artificielle

J'ai utilisé un assistant IA comme **tuteur technique** (explication de concepts bloquants) et pour des **relectures de code systématiques** après chaque étape, afin de repérer des incohérences que je n'aurais pas vues seul. Le choix de la stack, le modèle de données initial, l'intégration HTML/CSS et l'identification des règles métier de l'énoncé ont été faits en autonomie. Plusieurs suggestions de l'IA ont été ajustées ou rejetées quand elles ne correspondaient pas exactement à une règle de l'énoncé.

Document réalisé dans le cadre du TP Développeur Web et Web Mobile — Studi, session 2026.